使用方式與目錄

Compiler - Ch1-Ch3 考場短版 C++

2026-10-14｜完整八份 C++17｜只用標準函式庫
HTML 為使用者歷屆線索；其他為課程／作業延伸，非出題機率。
610 項檢查＋sanitizer；Linux 交叉編譯成功，Linux 實跑／DOMjudge Accepted 未驗證。

01 HTML 標籤配對 …… p.3 – 5

02 遞迴下降運算式 …… p.6 – 8

03 Scanner：沿用現有短程式 …… p.9 – 11

04 NFA 直接判定字串 …… p.12 – 13

05 HW5 固定四狀態 DFA …… p.14 – 15

06 ac 邊解析邊產 dc …… p.16 – 18

07 Stack VM：沿用現有短程式 …… p.19 – 20

08 LL(1)：較低優先補充 …… p.21 – 23

考場先確認

先抄題目需要的一份；每份獨立 main，不能一起連結。

只字串判定：HTML 用 html_check，NFA 用 nfa_check，固定 HW5 用 hw5_check。

輸出 AST/postfix：expression 第一行 MODE 改 ast/postfix；eval 的變數不合法。

ac 產碼：ac_dc 保留型別檢查與精確 decimal；要樹／摺疊改用原完整版本。

Scanner、VM、LL(1) 沿用直接流程；中文註解增加閱讀長度，不增加執行步驟。

HTML 引號內 > 不結束 tag；<p a=x/> 的 slash 屬於 value，p 仍未關閉。

HW5(a) accept state 可接下一個 a/b/d；HW5(d) accept 後再讀字元就拒絕。

案例數 T、EOF、空行、大小寫、輸出字樣、精度一律對齊原題。沒有原題就不能保證 Accepted。

所有程式只用標準 C++17。getline 字串題移除尾端 CR；整份 source 把 CR 視為空白。

用 .cpp 複製提交，PDF 長行只做視覺換行。完整版本仍保留 token/tree/text/DFA table 功能。

01 | HTML 標籤配對 | html_check.cpp

契約：整份 stdin 為一個 HTML fragment；只印 VALID／INVALID，兩種判定皆 exit 0。stack 保存未關閉標籤，END 必須配 top；空輸入合法。

範圍：保留原版 check 的屬性驗證、註解、DOCTYPE html、13 個 void 與 script/style/textarea/title raw text；名稱不分 ASCII 大小寫。明示關閉，普通元素 <x/> 採本題子集自閉合；非 browser DOM，不省略 end tag、不解碼 entity。

改法：改 main 的輸出字樣；一行一案才改成 getline 外層。配對時累積 stack.size() 可求最大深度。若要 token／tree／text，使用原 html_parser.cpp 的相應模式。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror html_check.cpp
```

輸入：

```
<div id="main"><p title="a>b">Hello <b>world</b></p><br><img src=x><!--ok--></div>
```

預設輸出：

```
VALID
```

完整程式：html_check.cpp

```cpp
// One HTML fragment -> VALID / INVALID. Explicit nesting, not browser recovery.
#include <iostream>
#include <iterator>
#include <set>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;
// 整份輸入保存在 s；p 是下一個尚未處理的 byte。只處理一份文件時不必重設 p。
string s;
size_t p = 0;
// 把 CR 與 LF 都視為空白，跨 Windows / Linux 的換行仍能解析。
bool space(char c) { return string(" \t\n\r\f").find(c) != string::npos; }
bool letter(char c) { return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z'); }
// 只轉 ASCII 大小寫：<DIV> 與 </div> 可配對，文字內容不會被改寫。
string lower(string t) { for (char& c : t) if (c >= 'A' && c <= 'Z') c += 'a' - 'A'; return t; }
// 統一用例外離開巢狀解析；不必在每層傳遞錯誤旗標。
void fail() { throw runtime_error("invalid HTML"); }
void skip() { while (p < s.size() && space(s[p])) ++p; }
// 讀 tag 或 attribute 名稱；先驗證第一字元，再讀完整名稱。
// 不能直接讀到空白：名稱內允許數字、-、_、:，但有自己的起始規則。
string name() {
    size_t start = p;
    if (p == s.size() || !(letter(s[p]) || s[p] == '_' || s[p] == ':')) fail();
    while (p < s.size() && (letter(s[p]) || (s[p] >= '0' && s[p] <= '9') ||
           s[p] == '-' || s[p] == '_' || s[p] == ':')) ++p;
    return lower(s.substr(start, p - start));
}
bool check() {
    // HTML void elements 沒有一般 end tag；它們不進 stack。
    // 本題另允許 <custom/> 自閉合，這是題目子集政策，非完整 browser 行為。
    const set<string> empty{"area","base","br","col","embed","hr","img","input",
                            "link","meta","source","track","wbr"};
    // raw text 中的 < 不一律是新 tag；要直接找對應的結束標籤。
    const set<string> raw{"script","style","textarea","title"};
    // stack 只保存「已開啟、尚未關閉」的 tag，由外層到內層排列。
    vector<string> stack;
    if (s.find('\0') != string::npos) fail();
    while (p < s.size()) {
        // 一般文字不影響巢狀關係，直接往後讀；遇到 < 才需要分類。
        if (s[p] != '<') { ++p; continue; }
        // 註解中的 <b> 不當成標籤；必須找到 -->，否則拒絕。
        if (s.compare(p, 4, "<!--") == 0) {
            size_t end = s.find("-->", p + 4);
            if (end == string::npos) fail();
            p = end + 3; continue;
        }
```

```
48            // DOCTYPE 是宣告，不推入 stack；本版只接受短形式 <!DOCTYPE html>。
49            if (lower(s.substr(p, 9)) == "<!doctype") {
50                p += 9;
51                if (p == s.size() || !space(s[p])) fail();
52                skip(); if (name() != "html") fail(); skip();
53                if (p == s.size() || s[p++] != '>') fail();
54                continue;
55            }
56            ++p;
57            // 剛跳過 <，下一字元 / 表示 end tag；不接受 < /a> 這種寫法。
58            bool end_tag = p < s.size() && s[p] == '/';
59            if (end_tag) ++p;
60            string tag = name();
61            // 結束標籤只能關閉最內層。例：<a><b></a> 不能先把 a 移除。
62            if (end_tag) {
63                skip();
64                if (p == s.size() || s[p++] != '>' || stack.empty() || stack.back() != tag) fail();
65                stack.pop_back(); continue;
66            }
67            // closed 表示不必再等待 end tag：void 或明示 />。
68            bool closed = empty.count(tag);
69            set<string> attrs;
70            while (true) {
71                // 每個 attribute 前要有空白；這個位置差用來拒絕 a="1"b="2"。
72                size_t before = p; skip();
73                if (p == s.size()) fail();
74                if (s[p] == '>') { ++p; break; }
75                if (s.compare(p, 2, "/>") == 0) { p += 2; closed = true; break; }
76                // 同一個 start tag 不允許重複 attribute；名稱已轉小寫。
77                if (p == before || !attrs.insert(name()).second) fail();
78                // 沒有 = 的 attribute 是 boolean attribute；回到名稱尾端，讓下輪驗證空白。
79                size_t after = p; skip();
80                if (p == s.size() || s[p] != '=') { p = after; continue; }
81                ++p; skip();
82                if (p == s.size()) fail();
83                // 引號內的 > 只是值的一部分；不可用第一個 > 草率切開 tag。
84                if (s[p] == '\'' || s[p] == '"') {
85                    char quote = s[p++];
86                    size_t end = s.find(quote, p);
87                    if (end == string::npos) fail();
88                    p = end + 1;
89                } else {
90                    size_t start = p;
91                    while (p < s.size() && !space(s[p]) && s[p] != '>') {
92                        if (string("\"'<=`").find(s[p]) != string::npos) fail();
93                        ++p;
94                    }
95                    // 無引號值至少有一個字元；a=x/> 的 / 是值的一部分，a=x /> 才自閉合。
96                    if (p == start) fail(); // Slash in an unquoted value is part of that value.
97                }
98            }
99            if (closed) continue;
100           // 一般未自閉合 START 才入 stack。求最大深度時就在這裡更新答案。
101           stack.push_back(tag);
102           if (raw.count(tag)) {
103               // 只找完整對應的 end tag；</scriptx> 不可誤認成 </script>。
104               size_t end = s.find("</", p);
105               while (end != string::npos) {
106                   size_t after = end + 2 + tag.size();
107                   while (after < s.size() && space(s[after])) ++after;
108                   if (lower(s.substr(end + 2, tag.size())) == tag && after < s.size() && s[after] ==
      '>') break;
109                   end = s.find("</", end + 2);
```

```
            }
            if (end == string::npos) fail();
            p = end;
        }
    }
    // 整份文件讀完仍有 tag 未關閉，就算沒有錯位 END 也要拒絕。
    return stack.empty();
}
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    // 以 EOF 當一案結尾，保留跨行 HTML；一行一案的題目需改這段。
    s.assign(istreambuf_iterator<char>(cin), {});
    // 讀取裝置失敗與文件語法非法不同：前者才用 exit 1。
    if (cin.bad()) { cerr << "input error\n"; return 1; }
    bool valid = false;
    // 語法錯誤也是分類結果 INVALID，所以這裡捕捉例外，不把它當程式崩潰。
    try { valid = check(); } catch (const runtime_error&) {}
    cout << (valid ? "VALID\n" : "INVALID\n");
    // 檢查輸出串流是否成功；不要印提示或 pause 訊息到 judge。
    return cout ? 0 : 1;
}
```

02 | 遞迴下降運算式 | expression.cpp

契約：一行一式；常數 MODE=eval（預設求值）、postfix 或 ast，修改這一行選題。+ - * /、括號、一元正負、十進位及 e/E；二元左結合。錯誤 stderr、exit 1，整份 stdout 暫存。

範圍：eval 使用 long double、15 位有效數字，拒絕零除／非有限數／變數；ID 僅用於 postfix/ast。嵌套括號與一元鏈深度 <=256；AST 字串拼接最壞 $O(n^2)$。長加減鏈用 while，不占深遞迴。

改法：只求值可刪 postfix 與 ast 相關行；只畫 AST 可刪 number 的計算。若嵌套超過 256 或 AST 很長，原版 iterative expression.cpp 保留相同運算規則。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror expression.cpp
```

輸入：

```
(2+3)*4
100-30-1
8/2/2
-2*-3
.5 + 1e-2
```

預設輸出：

```
20
69
2
6
0.51
```

完整程式：expression.cpp

```
// One expression per line. Change MODE to "postfix" or "ast" for those tasks.
#include <cmath>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;
// 選一種題目輸出：eval=數值；postfix=後序式；ast=括號 AST。
// 精度設定是有效數字，並非固定小數位；固定小數位要另外使用 fixed。
const string MODE = "eval";
string s;
size_t p;
// 每次完成子運算後才記錄 operator，因此順序自然是左、右、根。
vector<string> postfix;
// 回傳目前子運算式的表示；只在選中的模式建立 AST 或計算 number。
struct Value { string ast; long double number; };
bool digit(char c) { return c >= '0' && c <= '9'; }
bool letter(char c) { return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_'; }
void fail() { throw runtime_error("invalid expression"); }
// peek 先跳空白，所以每層文法不必重複處理空格與 CR。
void skip() { while (p < s.size() && string(" \t\r\n").find(s[p]) != string::npos) ++p; }
// EOF 以零字元回報；main 還會確認游標位置，拒絕內嵌 NUL。
char peek() { skip(); return p < s.size() ? s[p] : '\0'; }
Value expr(int depth);
// 遞迴下降（recursive descent）：每個函式對應一層優先序。
// 括號與連續一元正負會增加遞迴深度，長二元加減鏈則用迴圈。
Value unary(int depth) {
    // ponytail: nested parentheses/unary <= 256; use the iterative full version above that.
    if (depth > 256) fail();
    char c = peek();
    // 這裡出現的 + / - 位於 operand 前，是一元運算；-2 的 postfix 是 2 neg。
    if (c == '+' || c == '-') {
        ++p; Value v = unary(depth + 1);
        string op = c == '-' ? "neg" : "pos";
        postfix.push_back(op);
        if (MODE == "ast") v.ast = "(" + op + " " + v.ast + ")";
        if (c == '-') v.number = -v.number;
```

```
            return v;
        }
        // 括號內重新從 expr 開始；結束時必須消耗配對的 )。
        // 括號只控制結構，不會成為 AST 節點或 postfix operator。
        if (c == '(') {
            ++p; Value v = expr(depth + 1);
            if (peek() != ')') fail();
            ++p; return v;
        }
        size_t start = p;
        // 變數可以用於 AST / postfix；eval 沒有環境表，因此拒絕變數。
        bool id = letter(c);
        if (id) { while (p < s.size() && (letter(s[p]) || digit(s[p]))) ++p; }
        else {
            // 本題 number 允許 .5 與 10.；此規則與 ac 的必須有小數尾數不同。
            if (!digit(c) && !(c == '.' && p + 1 < s.size() && digit(s[p + 1]))) fail();
            while (p < s.size() && digit(s[p])) ++p;
            if (p < s.size() && s[p] == '.') { ++p; while (p < s.size() && digit(s[p])) ++p; }
            // e/E 後可有正負號，但必須至少一個數字；1e+ 不合法。
            if (p < s.size() && (s[p] == 'e' || s[p] == 'E')) {
                ++p; if (p < s.size() && (s[p] == '+' || s[p] == '-')) ++p;
                if (p == s.size() || !digit(s[p])) fail();
                while (p < s.size() && digit(s[p])) ++p;
            }
        }
        string text = s.substr(start, p - start);
        postfix.push_back(text);
        Value v{MODE == "ast" ? text : "", 0};
        // 數字必須完整轉換成功且為有限值；不能把 12abc 默默當作 12。
        if (MODE == "eval") {
            istringstream input(text);
            if (id || !(input >> v.number) || input.peek() != char_traits<char>::eof() || !isfinite(v.
    number)) fail();
        }
        return v;
    }
    // 先讀完右 operand，再合併左／右；op 的方向對減法、除法非常重要。
    Value combine(Value a, char op, Value b) {
        postfix.push_back(string(1, op));
        // ponytail: AST strings can cost O(n^2); use the arena version for huge AST output.
        if (MODE == "ast") a.ast = "(" + string(1, op) + " " + a.ast + " " + b.ast + ")";
        if (MODE == "eval") {
            if (op == '+') a.number += b.number;
            if (op == '-') a.number -= b.number;
            if (op == '*') a.number *= b.number;
            // 除法的右 operand 不可是零；結果溢位成無限大也拒絕。
            if (op == '/') { if (b.number == 0) fail(); a.number /= b.number; }
            if (!isfinite(a.number)) fail();
        }
        // AST 不保留括號 token；節點的父子關係已記錄優先序。
        return a;
    }
    // term 只處理乘除，先完成高優先序，再交回 expr。
    Value term(int depth) {
        Value v = unary(depth);
        while (peek() == '*' || peek() == '/') { char op = s[p++]; Value b = unary(depth); v = combine
    (v, op, b); }
        return v;
    }
    // while 每次用前次結果當左 operand，所以 100-30-1 是 (100-30)-1。
    Value expr(int depth) {
        Value v = term(depth);
        while (peek() == '+' || peek() == '-') { char op = s[p++]; Value b = term(depth); v = combine(
```

```
    v, op, b); }
101     return v;
102 }
103 int main() {
104     ios::sync_with_stdio(false); cin.tie(nullptr);
105     // 先暫存所有答案；其中一行有錯時，不送出前面已算的部分答案。
106     ostringstream out; out << setprecision(15);
107     try {
108         while (getline(cin, s)) {
109             // 每行是一案，游標與 postfix 必須清空，不能沿用上一案。
110             p = 0; postfix.clear(); Value v = expr(0);
111             // 成功解析前綴不代表整行合法；2(3)、1abc 或多出的 ) 仍要拒絕。
112             if (peek() != '\0' || p != s.size()) fail();
113             if (MODE == "eval") out << (v.number == 0 ? 0 : v.number) << '\n';
114             if (MODE == "ast") out << v.ast << '\n';
115             if (MODE == "postfix") {
116                 for (size_t i = 0; i < postfix.size(); ++i) out << (i ? " " : "") << postfix[i];
117                 out << '\n';
118             }
119         }
120         if (cin.bad()) fail();
121         cout << out.str(); return cout ? 0 : 1;
122     } catch (const exception& e) { cerr << e.what() << '\n'; return 1; }
123 }
```

03 | Scanner：沿用現有短程式 | lexer.cpp

契約：整份 stdin；印 KW/ID/NUM/STRING/CHAR/OP/PUNCT 與帶引號 lexeme，最後 EOF。忽略 // 與非巢狀 /* */。長運算子先比，完整 ID 再查 keyword。

範圍：C-like 補充，非完整 C/C++；ID [A-Za-z_][A-Za-z_0-9]*；number 支援 .5、10.、e/E；CHAR 單 byte 或單 escape。

改法：keyword 改 keywords；運算子改 ops；輸出改最後兩行。沿用原本順序掃描的邏輯，追加中文註解；不新增架構。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror lexer.cpp
```

輸入：

```
int sum=12; // skip
float x=1.2e-3; /*skip*/ sum+=2; print("a>b");
```

預設輸出：

```
KW "int"
ID "sum"
OP "="
NUM "12"
PUNCT ";"
KW "float"
ID "x"
OP "="
NUM "1.2e-3"
PUNCT ";"
ID "sum"
OP "+="
NUM "2"
PUNCT ";"
KW "print"
PUNCT "("
STRING "\"a>b\""
PUNCT ")"
PUNCT ";"
EOF
```

完整程式：lexer.cpp

```
// C-like exam lexer: identifiers, decimal/exponent, strings, comments, longest operators.
// Supplement identifier rule: [A-Za-z_][A-Za-z_0-9]* (not Ch3's restricted underscores).
#include <iomanip>
#include <iostream>
#include <iterator>
#include <set>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
// 採 ASCII 範圍比較，避免 signed char 直接傳給 cctype 造成問題。
bool digit(char c) { return c >= '0' && c <= '9'; }
// 此補充題的 ID 可用底線；不要直接套用到 Ch3 另有底線限制的原題。
bool letter(char c) { return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_'; }
bool space(char c) { return c == ' ' || c == '\t' || c == '\r' || c == '\n' || c == '\f' || c == '\v'; }
int main() {
    std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
    try {
        // 整份 source 讀到 EOF；不可用 cin >> s，否則空白後的內容會遺失。
        const std::string s{std::istreambuf_iterator<char>(std::cin), {}};
        if (std::cin.bad()) throw std::runtime_error("input read failed");
        // 先讀完整 ID 再查這張表：intx 是 ID，不是 int 接 x。
        const std::set<std::string> keywords{"int","float","double","char","bool","void","if","else","while","for","return","print"};
        // Longest match：三字元運算子放在二字元前；單字元最後判斷。
        const std::vector<std::string> ops{"<<=",">>=","++","--","==","!=","<=",">=","&&","||","+=","-=","*=","/=","%=","<<",">>","->","&=","|=","^=","::"};
        // 只在完整掃描成功後輸出 token，避免非法尾端造成半份輸出。
        std::ostringstream output;
```

```
28          for (std::size_t p = 0; p < s.size();) {
29              if (space(s[p])) { ++p; continue; }
30              // 註解先於 / 運算子；單行註解遇 CR 或 LF 都結束。
31              if (s.compare(p, 2, "//") == 0) { while (p < s.size() && s[p] != '\r' && s[p] != '\n') ++p; continue; }
32              // 區塊註解採非巢狀規則；找不到 */ 必須報錯。
33              if (s.compare(p, 2, "/*") == 0) {
34                  const auto end = s.find("*/", p + 2);
35                  if (end == std::string::npos) throw std::runtime_error("unclosed block comment");
36                  p = end + 2; continue;
37              }
38              // start 記住 token 起點；結束位置 p 可用來直接切原始 lexeme。
39              const auto start = p;
40              std::string kind;
41              // ID 一次讀到底, 再分成 keyword / identifier, 避免錯切前綴。
42              if (letter(s[p])) {
43                  ++p; while (p < s.size() && (letter(s[p]) || digit(s[p]))) ++p;
44                  kind = keywords.count(s.substr(start, p - start)) ? "KW" : "ID";
45              // 小數的點只有接數字才可當 number 起點；獨立 . 是 punctuation。
46              } else if (digit(s[p]) || (s[p] == '.' && p + 1 < s.size() && digit(s[p + 1]))) {
47                  kind = "NUM";
48                  while (p < s.size() && digit(s[p])) ++p;
49                  if (p < s.size() && s[p] == '.') { ++p; while (p < s.size() && digit(s[p])) ++p; }
50                  // 指數的正負號屬於 number；一般 -12 的 - 仍是獨立 operator。
51                  if (p < s.size() && (s[p] == 'e' || s[p] == 'E')) {
52                      ++p; if (p < s.size() && (s[p] == '+' || s[p] == '-')) ++p;
53                      if (p == s.size() || !digit(s[p])) throw std::runtime_error("missing exponent digits");
54                      while (p < s.size() && digit(s[p])) ++p;
55                  }
56              // 進入 literal 後, // 與 /* 不再當作註解；只找未跳脫的配對引號。
57              } else if (s[p] == '\'' || s[p] == '"') {
58                  const char quote = s[p++];
59                  // token 的類型與 lexeme 分開存；內容不需轉成執行期字串值。
60                  kind = quote == '"' ? "STRING" : "CHAR";
61                  int units = 0;
62                  while (p < s.size() && s[p] != quote) {
63                      if (s[p] == '\r' || s[p] == '\n' || s[p] == '\0') throw std::runtime_error("newline/NUL in literal");
64                      // 反斜線連同下一個字元一起吃掉；例如 \" 不會提早結束 string。
65                      if (s[p] == '\\') {
66                          ++p;
67                          if (p == s.size() || s[p] == '\r' || s[p] == '\n' || s[p] == '\0') throw std::runtime_error("incomplete escape");
68                      }
69                      ++p; ++units;
70                  }
71                  if (p == s.size()) throw std::runtime_error("unclosed literal");
72                  ++p;
73                  // CHAR 只允許一個 byte 或一個簡單 escape；不是完整 C 字元常量規範。
74                  if (quote == '\'' && units != 1) throw std::runtime_error("character literal needs one character/escape");
75              } else {
76                  // 先試完整多字元 operator, 再退到單字元；不要用固定切兩字元的做法。
77                  for (const auto& op : ops) if (s.compare(p, op.size(), op) == 0) { p += op.size(); kind = "OP"; break; }
78                  if (kind.empty()) {
79                      const char c = s[p++];
80                      // 此處才退到單字元 operator, 確保 += 不被切成 + 和 =。
81                      if (std::string("+-*/%=<>!&|^~?:").find(c) != std::string::npos) kind = "OP";
82                      else if (std::string("(){}[],;.").find(c) != std::string::npos) kind = "PUNCT";
83                      else throw std::runtime_error("invalid byte at " + std::to_string(start));
```

```
                    }
                }
                // std::quoted 是輸出包裝；lexeme 本身仍保留原始大小寫與引號。
                output << kind << ' ' << std::quoted(s.substr(start, p - start)) << '\n';
            }
            // 只有題目要求才保留額外 EOF token；考場可刪此行。
            output << "EOF\n";
            std::cout << output.str(); return std::cout ? 0 : 1;
        } catch (const std::exception& e) { std::cerr << "error: " << e.what() << '\n'; return 1; }
    }
```

04 | NFA 直接判定字串 | nfa_check.cpp

契約：n m start k；k 個接受 state；alphabet；m 條 from symbol to；其後一行一字串。state=0..n-1，#=epsilon，空行=空字串。印 ACCEPT／REJECT。

範圍：n<=256，m<=65536；只做 closure(move(states,c)) 的 NFA 模擬，不產 DFA 表。不再因完整 DFA 超過 4096 states 而失敗。

改法：字串辨識就用這份；若題目明確要求 subset construction／DFA 表／集合 trace，改用原 nfa_to_dfa.cpp，勿用字串判定代替表格作答。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror nfa_check.cpp
```

輸入：

```
6 7 0 1
5
bcd
0 # 1
0 # 4
1 b 2
2 c 3
3 # 1
3 # 4
4 d 5
d
bcd
bcbcd

bc
bd
bcdd
```

預設輸出：

```
ACCEPT
ACCEPT
ACCEPT
REJECT
REJECT
REJECT
REJECT
```

完整程式：nfa_check.cpp

```
// n m start k; k finals; alphabet; m edges (from symbol to); words, one per line.
#include <iostream>
#include <map>
#include <set>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;
// 用 set 去重：NFA 同一時間可能在多個 state；不能只用一個 int。
using States = set<int>;
// edges[from][symbol] 保存全部可達 state；# 專門表示 epsilon。
vector<map<char, States>> edges;
void fail() { throw runtime_error("invalid NFA input"); }
// epsilon-closure：不用讀字元也能走的邊，必須一直追到固定點。
States closure(States states) {
    // todo 是待展開 state；只有新插入的 state 才進 todo，避免 epsilon cycle 無限迴圈。
    vector<int> todo(states.begin(), states.end());
    while (!todo.empty()) {
        int from = todo.back(); todo.pop_back();
        for (int to : edges[from]['#']) if (states.insert(to).second) todo.push_back(to);
    }
    return states;
}
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
```

```
    try {
        // n=state 數, m=edge 數, start=起點, k=接受 state 數。
        int n, m, start, k;
        if (!(cin >> n >> m >> start >> k) || n < 1 || n > 256 || m < 0 || m > 65536 || k < 0 || k > n) fail();
        // state 使用 0-based；換成 1-based 時, 也要把 edges 容量加一。
        auto valid = [n](int state) { return state >= 0 && state < n; };
        if (!valid(start)) fail();
        States finals;
        for (int i = 0, state; i < k; ++i) if (!(cin >> state) || !valid(state) || !finals.insert(state).second) fail();
        string alphabet;
        // alphabet 不能重複也不能含 epsilon 符號；每個輸入字元才有明確合法性。
        if (!(cin >> alphabet) || alphabet.find('#') != string::npos ||
            set<char>(alphabet.begin(), alphabet.end()).size() != alphabet.size()) fail();
        // 先按 n 配好容量, 輸入每條 edge 時再驗證起終點索引。
        edges.resize(n);
        for (int i = 0; i < m; ++i) {
            int from, to; string symbol;
            if (!(cin >> from >> symbol >> to) || !valid(from) || !valid(to) || symbol.size() != 1 ||
                (symbol != "#" && alphabet.find(symbol[0]) == string::npos)) fail();
            // 同一 from / symbol 可以有多個 to；set 也會自動移除重複 edge。
            edges[from][symbol[0]].insert(to);
        }
        // 混用 >> 與 getline：先吃完轉移表最後一行, 才開始讀案例。
        string word; getline(cin, word);
        if (word.find_first_not_of(" \t\r") != string::npos) fail();
        while (getline(cin, word)) {
            // 只去掉 CRLF 的尾端 CR；不要刪普通空白, 因為空白可能是非法字元。
            if (!word.empty() && word.back() == '\r') word.pop_back();
            // 每案重新建立起始 closure；空行也需照樣檢查是否接受 epsilon。
            States states = closure({start});
            for (char c : word) {
                // 字母表外字元不接受；不嘗試跳過它以匹配剩餘前綴。
                if (alphabet.find(c) == string::npos) { states.clear(); break; }
                // move 收集目前所有 state 的 c 邊, 不先挑某一條路。
                States next;
                for (int from : states) next.insert(edges[from][c].begin(), edges[from][c].end());
                // 讀一字元後再展開 epsilon；否則會漏掉先 c 再 epsilon 的路徑。
                states = closure(next);
            }
            // 必須讀完整字串才判定；中途碰到 final 但之後走死仍然拒絕。
            bool accepted = false;
            for (int state : states) if (finals.count(state)) accepted = true;
            cout << (accepted ? "ACCEPT\n" : "REJECT\n");
        }
        if (cin.bad()) fail();
        return cout ? 0 : 1;
    } catch (const exception& e) { cerr << e.what() << '\n'; return 1; }
}
```

05 | HW5 固定四狀態 DFA | hw5_check.cpp

契約：一行一字串，空行拒絕；HW5A=true 辨識 (a|(bc)*d)+，改 false 辨識 (bc)*d。印 ACCEPT／REJECT，不需命令列參數。

範圍：只有固定兩個語言；0=start、1=剛讀 b、2=剛讀 bc、3=accept、-1=dead。這是精簡的辨識 DFA，不是投影片子集構造的原始 state sets。

改法：HW5A 常數選題；改 ACCEPT/REJECT 字樣。若要求原 NFA 邊、epsilon closure 或 subset DFA 表，用 automata_homework.cpp 5a/5d --table。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror hw5_check.cpp
```

輸入：

```
a
d
ad
bcd
bcbcd
abcbcdad
b
bc
abca
bcdd
```

預設輸出：

```
REJECT
ACCEPT
ACCEPT
ACCEPT
ACCEPT
ACCEPT
ACCEPT
REJECT
REJECT
REJECT
ACCEPT
```

完整程式：hw5_check.cpp

```cpp
// Fixed DFA for HW5(a), or HW5(d) after changing HW5A to false.
#include <iostream>
#include <string>
using namespace std;
// true 對應 (a|(bc)*d)+；false 對應 (bc)*d，不需命令列參數。
const bool HW5A = true;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    string word;
    // 空行是案例；不要用 >> word，否則 epsilon 測例會被略過。
    while (getline(cin, word)) {
        // getline 已吃掉 LF；CRLF 留下的尾端 CR 要移除。
        if (!word.empty() && word.back() == '\r') word.pop_back();
        // 0=start, 1=after b, 2=after bc, 3=accept, -1=dead.
        // 每行從起點重新開始；接受條件只看讀完後的 state。
        int state = 0;
        for (char c : word) {
            // 死狀態不可復活；字串剩餘內容已不可能改成接受。
            if (state == -1) break;
            // 剛讀 b 時只能接 c，才形成完整 bc；bd 或 ba 都拒絕。
            if (state == 1) state = c == 'c' ? 2 : -1;
            // 完整 bc 後可再接 b 開始另一組 bc，或接 d 結束該段。
            else if (state == 2) state = c == 'b' ? 1 : c == 'd' ? 3 : -1;
            // HW5(d) 只有一段；已讀終止 d 後再出現任何字元都拒絕。
            else if (state == 3 && !HW5A) state = -1;
            // 起點／HW5(a) 已接受處：a 或 d 可完成一段；b 進入等待 c。
            else state = c == 'b' ? 1 : c == 'd' || (HW5A && c == 'a') ? 3 : -1;
        }
```

```
            // 空字串保持 state 0，所以兩個原語言都拒絕 epsilon。
            cout << (state == 3 ? "ACCEPT\n" : "REJECT\n");
        }
        if (cin.bad()) { cerr << "input error\n"; return 1; }
        // stdout 僅包含每案答案；不印轉移表，除非題目另有要求。
        return cout ? 0 : 1;
    }
```

06 | ac 邊解析邊產 dc | ac_dc.cpp

契約：整份 stdin 為 ac 程式；f/float、i/int 宣告在前，p 印變數，單字母 ID，加減左結合。保留重複／未宣告／float→int 檢查，正確時只印 dc code。

範圍：支援小數與 lowercase e signed exponent；帶 . 或 e 就是 float。精確移動小數點，exponent 絕對值及 mantissa 長度 <=10000；無括號／乘除／註解。print 使用 l<變數>/p/si 形式。

改法：int→float 時印 5 k；assign 後印 0 k。value 直接回 leaf code 與型別，主迴圈直接產碼，沒有樹走訪。題目要求 parse tree／AST／常數摺疊時，用原 ac_homework.cpp 的相應模式。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror ac_dc.cpp
```

輸入：

```
i x
x=100
x=x+30+1
p x
```

預設輸出：

```
100
sx
0 k
lx
30
+
1
+
sx
0 k
lx
p
si
```

完整程式：ac_dc.cpp

```
// ac -> dc, with type checks. No trees or constant folding in this short version.
#include <array>
#include <iostream>
#include <iterator>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>
using namespace std;
// kind 壓成一字元：i/f=宣告、p=print、v=單字母變數、n=number、$=EOF。
struct Token { char kind; string text; };
vector<Token> tokens;
size_t p = 0;
// 符號表（symbol table）直接用字母偏移；0 表示未宣告。
array<char, 26> types{}; // 0=undeclared, i=int, f=float.
void fail() { throw runtime_error("invalid ac program"); }
// 負數不作為 leaf literal；原 ac 要寫 0-數字，e 的負號則屬於 exponent。
bool digit(char c) { return c >= '0' && c <= '9'; }
// 先 tokenize 整份 ac；沒有分號，宣告與敘述的邊界由 token kind 判定。
void scan(const string& s) {
    for (size_t j = 0; j < s.size();) {
        if (string(" \t\r\n\v\f").find(s[j]) != string::npos) { ++j; continue; }
        size_t start = j;
        char kind = s[j];
        if (digit(s[j])) {
            kind = 'n';
            while (j < s.size() && digit(s[j])) ++j;
            // ac 小數要求點後有數字；1. 不合法，與 expression 補充文法不同。
            if (j < s.size() && s[j] == '.') {
                ++j; if (j == s.size() || !digit(s[j])) fail();
                while (j < s.size() && digit(s[j])) ++j;
```

```
                }
                // 科學記號 e 後可帶正負；即使 1e0 數值是整數，型別仍是 float。
                if (j < s.size() && s[j] == 'e') {
                    ++j; if (j < s.size() && (s[j] == '+' || s[j] == '-')) ++j;
                    if (j == s.size() || !digit(s[j])) fail();
                    while (j < s.size() && digit(s[j])) ++j;
                }
            } else {
                ++j;
                // 保留教材一字母 ID 與最長 keyword 前綴：floatx 切 float + x。
                if (kind == 'f' && s.compare(start, 5, "float") == 0) j = start + 5;
                else if (kind == 'i' && s.compare(start, 3, "int") == 0) j = start + 3;
                else if ((kind >= 'a' && kind <= 'z') || kind == '=' || kind == '+' || kind == '-') {}
                else fail();
                if (kind >= 'a' && kind <= 'z' && kind != 'i' && kind != 'f' && kind != 'p') kind = 'v
';
            }
            tokens.push_back({kind, s.substr(start, j - start)});
        }
        // 加入 EOF sentinel，parser 可一直查看下一個 token，不必另猜輸入是否結束。
        tokens.push_back({'$', ""});
    }
    // dc 不直接接受 e 記號；用字串移動小數點，避免 double 先捨入。
    string decimal(string text) {
        int exponent = 0;
        size_t e = text.find('e');
        if (e != string::npos) {
            size_t j = e + 1;
            bool negative = text[j] == '-';
            if (text[j] == '+' || text[j] == '-') ++j;
            for (; j < text.size(); ++j) {
                // 限制 exponent 絕對值 10000，且累積前檢查，避免 int overflow 與巨大配置。
                if (exponent > (10000 - (text[j] - '0')) / 10) fail();
                exponent = exponent * 10 + text[j] - '0';
            }
            if (negative) exponent = -exponent;
            text.resize(e);
        }
        if (text.size() > 10000) fail();
        size_t dot = text.find('.');
        // point 是移除小數點後，新的小數位置：原位置加 exponent。
        int point = static_cast<int>(dot == string::npos ? text.size() : dot) + exponent;
        if (dot != string::npos) text.erase(dot, 1);
        // 往左超過頭：補 0. 與零；往右超過尾：補尾零；中間：插回小數點。
        if (point <= 0) text = "0." + string(-point, '0') + text;
        else if (point >= static_cast<int>(text.size())) text.append(point - text.size(), '0');
        else text.insert(point, ".");
        size_t first = text.find_first_not_of('0');
        if (first == string::npos) return "0";
        return text.substr(text[first] == '.' ? first - 1 : first);
    }
    // id 只接受單一小寫字母變數 token；i/f/p 已是保留字。
    char id() {
        if (tokens[p].kind != 'v') fail();
        return tokens[p++].text[0];
    }
    // 查詢型別同時拒絕未宣告變數；沒有檢查變數是否曾賦值（沿用課程規則）。
    bool floating(char c) { if (!types[c - 'a']) fail(); return types[c - 'a'] == 'f'; }
    // leaf 回傳 pair：first 是 target code，second 是是否 float。
    pair<string, bool> value() {
        if (tokens[p].kind == 'v') {
            char c = id(); return {"l" + string(1, c) + "\n", floating(c)};
        }
```

06 | ac 邊解析邊產 dc | ac_dc.cpp

```
        if (tokens[p].kind != 'n') fail();
        string text = tokens[p++].text;
        // 帶 . 或 e 的 number 為 float；decimal 只改表示，不改數學值。
        return {decimal(text) + "\n", text.find_first_of(".e") != string::npos};
    }
    int main() {
        ios::sync_with_stdio(false); cin.tie(nullptr);
        try {
            string source{istreambuf_iterator<char>(cin), {}};
            if (cin.bad()) fail();
            scan(source);
            // 即使前段已產碼，也先不送出；尾端型別／語法錯誤時整份 stdout 仍為空。
            ostringstream out;
            // 宣告必須全部在敘述前；同一變數宣告兩次會拒絕。
            while (tokens[p].kind == 'i' || tokens[p].kind == 'f') {
                char type = tokens[p++].kind, c = id();
                if (types[c - 'a']) fail();
                types[c - 'a'] = type;
            }
            // 因為沒有分號，一段加減後看到 id / p 就進入下一個 statement。
            while (tokens[p].kind != '$') {
                // dc 的 p 不 pop，所以印完用 si 移走頂端；i 不會與 ac 變數衝突。
                if (tokens[p].kind == 'p') {
                    ++p; char c = id(); floating(c);
                    out << 'l' << c << "\np\nsi\n"; continue;
                }
                char target = id(); bool target_float = floating(target);
                if (tokens[p++].kind != '=') fail();
                // 直接輸出 operand；不建 AST。迴圈左結合，所以 x-30-1 不會變成 x-(30-1)。
                auto left = value(); out << left.first;
                while (tokens[p].kind == '+' || tokens[p].kind == '-') {
                    char op = tokens[p++].kind; auto right = value();
                    // 左側已累積的 int 結果需要提升：先印 conversion，再印右側 code。
                    if (!left.second && right.second) out << "5 k\n";
                    out << right.first;
                    // 右側 int 需要提升：先印右側 code，再印 conversion；順序不能顛倒。
                    if (left.second && !right.second) out << "5 k\n";
                    out << op << '\n'; left.second = left.second || right.second;
                }
                // float 不能指定給 int；int 指定給 float 可提升。全部成功才送出 out。
                if (!target_float && left.second) fail();
                if (target_float && !left.second) out << "5 k\n";
                // s<變數> pop 並存暫存器；assign 完把 scale 設回 0。
                out << 's' << target << "\n0 k\n";
            }
            cout << out.str(); return cout ? 0 : 1;
        } catch (const exception& e) { cerr << e.what() << '\n'; return 1; }
    }
```

07 | Stack VM：沿用現有短程式 | stack_vm.cpp

契約：一行一指令：PUSH、LOAD、STORE、ADD/SUB/MUL/DIV、NEG、DUP、PRINT；PRINT/STORE pop。long double，15 位有效數字；錯誤 stderr、exit 1，暫存 stdout。

範圍：無條件／跳躍／函式；只做 stack interpreter，不是 dc。LOAD 未定義、underflow、div0 拒絕。

改法：先 pop 右 b，再左 a，算 a op b；PRINT 若題目要求不 pop，就改讀 stack.back()。沿用原本直接執行流程，追加中文註解。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror stack_vm.cpp
```

輸入：

```
PUSH 100
STORE x
LOAD x
PUSH 30
PUSH 1
ADD
ADD
STORE x
LOAD x
PRINT
```

預設輸出：

```
131
```

完整程式：stack_vm.cpp

```
// Tiny stack interpreter. One instruction per line; long double arithmetic.
// PUSH number, LOAD name, STORE name, ADD/SUB/MUL/DIV/NEG, DUP, PRINT.
#include <cmath>
#include <iomanip>
#include <iostream>
#include <locale>
#include <map>
#include <sstream>
#include <stdexcept>
#include <string>
#include <vector>
// 變數名稱首字元是字母或底線，其後才可出現數字。
bool name_valid(const std::string& name) {
    if (name.empty()) return false;
    for (std::size_t i = 0; i < name.size(); ++i) {
        const char c = name[i];
        if (!((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_' || (i && c >= '0' && c <= '9'))) return false;
    }
    return true;
}
int main() {
    std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
    try {
        // 用 vector.back 當 stack 頂端；沒有固定容量，不必自寫 linked list。
        std::vector<long double> stack;
        // vars 存變數；LOAD 不使用下標默認 0，未定義必須拒絕。
        std::map<std::string, long double> vars;
        // 顯示 15 位有效數字；資料仍保留 long double，不先轉 double。
        std::ostringstream output; output.imbue(std::locale::classic()); output << std::setprecision(15);
        // 共用 pop 先檢查空 stack，避免對空 vector 呼叫 back。
        const auto pop = [&] { if (stack.empty()) throw std::runtime_error("stack underflow"); const auto value = stack.back(); stack.pop_back(); return value; };
        std::string line; int line_number = 0;
        while (std::getline(std::cin, line)) {
            ++line_number;
            // 每行一條指令：先讀 op，再檢查需要的參數和多餘參數。
            std::istringstream input(line); input.imbue(std::locale::classic());
```

```
            std::string op, arg, extra;
            if (!(input >> op)) continue;
            // PUSH/LOAD/STORE 恰好需要一個參數；ADD 等不可多帶 argument。
            const bool needs_arg = op == "PUSH" || op == "LOAD" || op == "STORE";
            if (needs_arg && !(input >> arg)) throw std::runtime_error("missing argument at line " + std::to_string(line_number));
            if (input >> extra) throw std::runtime_error("extra argument at line " + std::to_string(line_number));
            // 數字必須完整解析且有限；1abc 不可偷偷讀成 1。
            if (op == "PUSH") {
                std::istringstream number(arg); number.imbue(std::locale::classic()); long double value;
                if (!(number >> value) || number.peek() != std::char_traits<char>::eof() || !std::isfinite(value)) throw std::runtime_error("invalid finite number");
                stack.push_back(value);
            } else if (op == "LOAD" || op == "STORE") {
                if (!name_valid(arg)) throw std::runtime_error("invalid variable name");
                // STORE 移走頂端存入變數；LOAD 只複製已存在的變數值。
                if (op == "STORE") vars[arg] = pop();
                else { const auto it = vars.find(arg); if (it == vars.end()) throw std::runtime_error("undefined variable " + arg); stack.push_back(it->second); }
            // 本題 PRINT 會 pop；題目若要像 dc 的 p 不 pop，改成讀 back 而不移除。
            } else if (op == "PRINT") { const auto value = pop(); output << (value == 0 ? 0 : value) << '\n'; }
            // NEG 是一元指令，只取一個值；不能與 SUB 的兩個 operand 混用。
            else if (op == "NEG") { const auto value = pop(); stack.push_back(-value); }
            // DUP 把頂端變成兩份；這裡先 pop 再 push 兩次，淨增加一項。
            else if (op == "DUP") { const auto value = pop(); stack.push_back(value); stack.push_back(value); }
            else if (op == "ADD" || op == "SUB" || op == "MUL" || op == "DIV") {
                // postfix 先取右 operand b，再取左 a；8 2 SUB 必須得到 6。
                const auto b = pop(), a = pop(); long double result;
                if (op == "ADD") result = a + b;
                else if (op == "SUB") result = a - b;
                else if (op == "MUL") result = a * b;
                else { if (b == 0) throw std::runtime_error("division by zero"); result = a / b; }
                // 即使 operand 有限，乘法仍可能溢位；結果也要檢查。
                if (!std::isfinite(result)) throw std::runtime_error("non-finite result");
                stack.push_back(result);
            } else throw std::runtime_error("unknown instruction " + op);
        }
        if (std::cin.bad()) throw std::runtime_error("input read failed");
        // 所有 PRINT 暫存到最後；後面有錯誤時不輸出半份執行結果。
        std::cout << output.str(); return std::cout ? 0 : 1;
    } catch (const std::exception& e) { std::cerr << "error: " << e.what() << '\n'; return 1; }
}
```

08 | LL(1)：較低優先補充 | ll1.cpp

契約：P 條產生式，一行 A -> X Y，# 單獨為 epsilon；預設印 FIRST/FOLLOW/SELECT、table 與 conflict。--check 判定其後每行 token sequence。

範圍：通用表格功能需要固定點、nullable suffix 與衝突檢查；保留原本固定點演算法，追加中文註解。P<=1000；parse<=1000000 步；不消除左遞迴或做 left factoring。正式 FIRST/FOLLOW 範圍未確認。

改法：若已知題目固定文法，只寫對應 recursive descent 更好記；若要求任意 CFG 的 FIRST/FOLLOW，保留本版以免刪掉 nullable 的必要處理。DOMjudge 不傳 args 時固定 main 的 mode。

```
Compile: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror ll1.cpp
```

輸入：

```
4
E -> T R
R -> + T R
R -> #
T -> num
```

預設輸出：

```
FIRST(E)={num} FOLLOW(E)={$}
FIRST(R)={#,+} FOLLOW(R)={$}
FIRST(T)={num} FOLLOW(T)={$,+}
SELECT(1)={num}
SELECT(2)={+}
SELECT(3)={$}
SELECT(4)={num}
M[E,num]=1
M[R,$]=3
M[R,+]=2
M[T,num]=4
LL1 YES
```

完整程式：ll1.cpp

```
// FIRST / FOLLOW / SELECT / LL(1) table. P productions, each "A -> X Y".
// One alternative per line; '#' alone is epsilon; '$' is reserved EOF.
// Start symbol = first LHS. --check reads whitespace-separated token sequences afterward.
#include <iostream>
#include <map>
#include <set>
#include <sstream>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>
// 符號採字串，所以 num、Expr、ExprTail 不必改成單字元代號。
using Set = std::set<std::string>;
struct Rule { std::string lhs; std::vector<std::string> rhs; };
// 計算整段 RHS 的 FIRST，不是只看第一字元。
// 如果前方可為 epsilon，還要繼續合併下一符號的 FIRST。
Set first_sequence(const std::vector<std::string>& rhs, std::size_t start, const std::map<std::string, Set>& first) {
    Set result;
    for (std::size_t i = start; i < rhs.size(); ++i) {
        // first 表只含 nonterminal；找不到表示 terminal，自己就是 FIRST。
        const auto it = first.find(rhs[i]);
        if (it == first.end()) { result.insert(rhs[i]); return result; }
        for (const auto& t : it->second) if (t != "#") result.insert(t);
        if (!it->second.count("#")) return result;
    }
    // 整段 RHS 都 nullable，或 suffix 是空串，FIRST 才包含 epsilon。
    result.insert("#"); return result;
}
// 只有集合真的增加才回傳 true；用這個旗標決定固定點迴圈能否停止。
bool merge(Set& target, const Set& source) { const auto before = target.size(); target.insert(source.begin(), source.end()); return target.size() != before; }
```

```cpp
void print_set(const Set& s) { std::cout << '{'; bool first = true; for (const auto& t : s) { if (!first) std::cout << ','; std::cout << t; first = false; } std::cout << '}'; }
int main(int argc, char** argv) {
    std::ios::sync_with_stdio(false); std::cin.tie(nullptr);
    try {
        const std::string mode = argc == 1 ? "--table" : argv[1];
        if (argc > 2 || (mode != "--table" && mode != "--check")) throw std::runtime_error("usage: ll1 [--table|--check]");
        int count; std::string line;
        if (!(std::cin >> count) || count < 1 || count > 1000) throw std::runtime_error("production count must be 1..1000");
        std::getline(std::cin, line);
        if (line.find_first_not_of(" \t\r") != std::string::npos) throw std::runtime_error("production starts on next line");
        std::vector<Rule> rules; Set nonterminals;
        // 一行一條 production、同一 LHS 的替代規則分開寫；不要直接輸入 |。
        for (int i = 0; i < count; ++i) {
            if (!std::getline(std::cin, line)) throw std::runtime_error("missing production");
            std::istringstream input(line); Rule rule; std::string arrow, symbol;
            if (!(input >> rule.lhs >> arrow) || arrow != "->" || rule.lhs == "#" || rule.lhs == "$" || rule.lhs == "|") throw std::runtime_error("expected A -> symbols");
            while (input >> symbol) {
                if (symbol == "$" || symbol == "|") throw std::runtime_error("reserved symbol; use one alternative per line");
                rule.rhs.push_back(symbol);
            }
            // 不允許省略 epsilon 記號；輸入空 RHS 必須明寫 #。
            if (rule.rhs.empty()) throw std::runtime_error("write # for epsilon");
            // # 只可單獨表示 epsilon；內部改用空 vector 表示空 RHS。
            if (rule.rhs.size() == 1 && rule.rhs[0] == "#") rule.rhs.clear();
            else for (const auto& s : rule.rhs) if (s == "#") throw std::runtime_error("epsilon must be alone");
            nonterminals.insert(rule.lhs); rules.push_back(std::move(rule));
        }
        // 先收齊所有 LHS 才知道 nonterminal，避免尚未讀到的規則被當 terminal。
        std::map<std::string, Set> first, follow;
        for (const auto& nt : nonterminals) { first[nt] = {}; follow[nt] = {}; }
        // FIRST 與 FOLLOW 都反覆掃規則到集合不再增加；支援循環相依。
        bool changed;
        do {
            changed = false;
            for (const auto& rule : rules) if (merge(first[rule.lhs], first_sequence(rule.rhs, 0, first))) changed = true;
        } while (changed);
        // 第一條 production 的 LHS 是 start；start 的 FOLLOW 包含 EOF $。
        follow[rules[0].lhs].insert("$");
        do {
            changed = false;
            for (const auto& rule : rules) for (std::size_t i = 0; i < rule.rhs.size(); ++i) {
                if (!nonterminals.count(rule.rhs[i])) continue;
                // 對 A -> α B β：把 FIRST(β) 去掉 epsilon 後加進 FOLLOW(B)。
                auto suffix = first_sequence(rule.rhs, i + 1, first);
                const bool nullable = suffix.erase("#") != 0;
                if (merge(follow[rule.rhs[i]], suffix)) changed = true;
                // β 可空時，再把 FOLLOW(A) 加進 FOLLOW(B)，不能漏掉這個傳播。
                if (nullable && merge(follow[rule.rhs[i]], follow[rule.lhs])) changed = true;
            }
        } while (changed);
        // SELECT = FIRST(RHS) 去 epsilon；若 RHS nullable，再加 FOLLOW(LHS)。
        std::vector<Set> select;
        std::map<std::pair<std::string, std::string>, std::vector<int>> table;
        bool ll1 = true;
        for (std::size_t i = 0; i < rules.size(); ++i) {
```

```cpp
            auto s = first_sequence(rules[i].rhs, 0, first);
            if (s.erase("#")) merge(s, follow[rules[i].lhs]);
            select.push_back(s);
            // 表格同一格若有兩條 production 就是 conflict；不可隨便選第一條。
            for (const auto& token : s) { auto& cell = table[{rules[i].lhs, token}]; cell.push_back(static_cast<int>(i)); if (cell.size() > 1) ll1 = false; }
        }
        if (mode == "--table") {
            for (const auto& nt : nonterminals) {
                std::cout << "FIRST(" << nt << ")="; print_set(first[nt]); std::cout << " FOLLOW(" << nt << ")="; print_set(follow[nt]); std::cout << '\n';
            }
            for (std::size_t i = 0; i < rules.size(); ++i) { std::cout << "SELECT(" << i + 1 << ")="; print_set(select[i]); std::cout << '\n'; }
            for (const auto& cell : table) {
                std::cout << "M[" << cell.first.first << ',' << cell.first.second << "]=";
                for (std::size_t i = 0; i < cell.second.size(); ++i) std::cout << (i ? "," : "") << cell.second[i] + 1;
                if (cell.second.size() > 1) std::cout << " CONFLICT";
                std::cout << '\n';
            }
            std::cout << (ll1 ? "LL1 YES\n" : "LL1 NO\n");
        } else {
            // 只能用無衝突的 LL(1) 表做這份 predictive parser。
            if (!ll1) throw std::runtime_error("grammar has LL(1) conflicts");
            while (std::getline(std::cin, line)) {
                std::istringstream input(line); std::vector<std::string> words; std::string word;
                bool accepted = true;
                while (input >> word) { if (word == "$" || word == "#" || nonterminals.count(word)) accepted = false; words.push_back(word); }
                // 輸入自動補 EOF；解析 stack 初始是 $ 與 start，頂端在 vector 尾端。
                words.push_back("$"); std::vector<std::string> stack{"$", rules[0].lhs}; std::size_t pos = 0, steps = 0;
                while (accepted && !stack.empty()) {
                    // ponytail: a finite study budget prevents pathological expansion; raise for larger judges.
                    if (++steps > 1000000) throw std::runtime_error("parse exceeds 1000000 steps");
                    // vector 尾端就是 stack top；push RHS 的順序要反過來。
                    const auto top = stack.back(); stack.pop_back();
                    // terminal 必須匹配 lookahead，才前進輸入位置；nonterminal 只展開，不吃字元。
                    if (!nonterminals.count(top)) { if (top != words[pos]) accepted = false; else ++pos; }
                    else {
                        const auto it = table.find({top, words[pos]});
                        if (it == table.end()) { accepted = false; continue; }
                        // 選到的 RHS 要反向 push，讓最左符號先被 pop／處理。
                        const auto& rhs = rules[it->second[0]].rhs;
                        for (auto r = rhs.rbegin(); r != rhs.rend(); ++r) stack.push_back(*r);
                    }
                }
                // stack 用完且輸入也全用完才接受；只匹配前綴不算成功。
                std::cout << (accepted && pos == words.size() ? "ACCEPT\n" : "REJECT\n");
            }
        }
        if (std::cin.bad()) throw std::runtime_error("input read failed");
        return std::cout ? 0 : 1;
    } catch (const std::exception& e) { std::cerr << "error: " << e.what() << '\n'; return 1; }
}
```